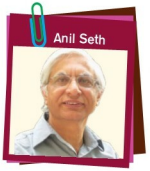

Exploring Big Data on a Desktop, with Hadoop, Elasticsearch and Pig

In continuation of earlier articles, the author goes further into the subject to discuss Elasticsearch and Pig, and explain how they can be used to create an index for a large number of PDF files.

If the files and data are already in Hadoop HDFS, is Elasticsearch still useful? How does one create an index?

Consider a large number of PDF files that need to be searched. As a first step, process each PDF file and store it as a record in an HDFS file. Then, you may experiment with two different but very simple approaches to create an index.

- Write a simple Python mapper using MapReduce streaming to create an index.
- Install the Elasticsearch-Hadoop plugin and create an index using a Pig script.

The environment for these experiments will be the same as in the earlier articles – three virtual machines, h-mstr, h-slv1 and h-slv2, each running HDFS and Elasticsearch services.

Loading PDF files into Hadoop HDFS

Enter the following code in *load_pdf_files.py*. Each PDF file is converted to a single line of text. Any tab characters are filtered so that there are no ambiguities when using a Pig script. For each file, the output will be the path, tab, file name and the text content of the file.

```
#!/usr/bin/python
from __future__ import print_function
import sys
import os
import subprocess
# Call pdftotext to convert the pdf file and store the result
in /tmp/pdf.txt
def pdf_to_text(inpath,infile):
    exit_code=subprocess.call(['pdftotext','/'.
join([inpath,infile]),'/tmp/pdf.txt'],stderr=ErrFile)
    return exit_code,'/tmp/pdf.txt'
# Join all the lines of the converted pdf file into a single
string
# Replace any tabs in the converted documents
# Write the file as a single line prefixing it with the path
and the name
def process_file(p,f):
    exit_code,textfile = pdf_to_text(p,f)
    if exit_code == 0:
        print("%s\t%s"%(p,f), end='\t')
        print("%s"% ' '.join([line.strip().replace('\t',' ')
for line in open(textfile)]))
# Generator for yielding pdf files
def get_documents(path):
    for curr_path,dirs,files in os.walk(path):
        for f in files:
            try:
                if f.rsplit('.',1)[1].lower() == 'pdf'
                    yield curr_path,f
            except:
                pass
# Start here
# Search for each file in the current path of type 'pdf' and
process it
try:
    path=sys.argv[1]
except IndexError:
    path='.'
# Use an error file for stderr to prevent these messages going
to hadoop streaming
ErrFile = open('/tmp/err.txt','w')
for p,f in get_documents(path):
    process_file(p,f)
```

Now, you can run the above program on your desktop and load data into a file in Hadoop HDFS as follows:

```
$  ./load_pdf_files.py  ~/Documents |HADOOP_USER_NAME=fedora \
 hdfs  dfs  -fs  hdfs://h-mstr/  -put  -  document_files.txt
```

Using MapReduce to create an index

Log into *h-mstr* as user *fedora* and enter the following code in *'indexing_mapper.py'*.

```
#!/usr/bin/python
import sys
from elasticsearch import Elasticsearch
# Generator for yielding each line split into path, file name
and the text content
def hdfs_input(sep='\t'):
  for line in sys.stdin:
    path,name,text=line[:-1].split(sep)
    yield path,name,text
# Create an index pdfdocs with fields path, title and text.
# Index each line received from Hadoop streaming
```